Лекція 5. Основи CSS та JavaScript:

від витоків до сучасності

Вступ

Світ веб-розробки стрімко розвивається. З кожним роком з’являються нові інструменти, фреймворки та технології, які полегшують роботу розробників і відкривають нові можливості у створенні сайтів. Проте, як і в будь-якій справі, найважливішим є ґрунтовне розуміння основ. Ця лекція створена для тих, хто робить перші кроки у веб-розробці, а також для тих, хто хоче краще зрозуміти зв’язок між CSS, JavaScript, препроцесорами, трансляторами та інструментами автоматизації.

Ми пройдемо шлях від історії виникнення стилів і скриптів на веб-сторінках до сучасних концепцій і практичних підходів. Лекція поділена на кілька логічних блоків:

1. Історія CSS і JavaScript
2. Синтаксис і основи CSS
3. Синтаксис і основи JavaScript
4. CSS-препроцесори
5. Транслятори JavaScript
6. Інструменти автоматизації та збірки
7. Підсумок і подальші кроки

1. Історія CSS

1.1 Як все починалося

На початку розвитку Інтернету веб-сторінки були надзвичайно простими. Вони складалися лише з тексту й елементарної структури HTML. Усі стилі задавалися прямо в HTML-документі за допомогою тегів, як-от <font>, , або атрибутів bgcolor, width тощо. Це ускладнювало підтримку сайту, особливо якщо він мав сотні або тисячі сторінок.

1.2 Виникнення CSS

У 1994 році розробник з Норвегії — Гокон Віум Лі (Håkon Wium Lie), який працював у W3C, запропонував створити окрему мову для опису стилю веб-сторінок. Ця ідея була підтримана — і в 1996 році з'явилася перша специфікація CSS 1.

1.3 Основні цілі CSS

CSS (Cascading Style Sheets — Каскадні Таблиці Стилів) був створений з чіткими цілями:

- Відокремити вміст (HTML) від зовнішнього вигляду
- Зробити сайти легшими у підтримці
- Дозволити змінювати стилі глобально — в одному місці
- Забезпечити адаптивність зовнішнього вигляду на різних пристроях

1.4 Подальший розвиток

- CSS 2 (1998): додано медіа-запити, позиціювання елементів, псевдокласи
- CSS 2.1 (2004): вдосконалено, але реалізовано не повністю через різну підтримку браузерами
- CSS3 (2011 – дотепер): модульна структура, гнучкі бокси (Flexbox), Grid Layout, анімації, трансформації

Сьогодні CSS — це потужний інструмент, який дозволяє створювати гнучкі, адаптивні, анімовані та сучасні інтерфейси.

2. Історія JavaScript

2.1 Чому з'явився JavaScript?

HTML надавав лише структуру, а CSS — лише зовнішній вигляд. Але користувачі потребували інтерактивності: перевірки форм, анімацій, динамічного завантаження даних. Саме тому виникла потреба у мові програмування, яка працює прямо в браузері.

2.2 Народження JavaScript

У 1995 році компанія Netscape Communications доручила Брендану Айку створити таку мову. Усього за 10 днів він розробив прототип, який спочатку називався LiveScript, а згодом був перейменований у JavaScript — з маркетингових міркувань (на той час популярною була мова Java, і компанія хотіла "підв'язати" себе до неї).

2.3 Хронологія розвитку

- 1995: Поява JavaScript у Netscape Navigator 2.0

- 1996: Поява JScript у браузері Internet Explorer
- 1997: Перший стандарт ECMAScript (ES1)
- 2009: Випуск Node.js — запуск JavaScript поза браузером
- 2015: Великий прорив — ECMAScript 6 (ES6), що додав let, const, класи, стрілочні функції, модулі
- 2020–сьогодні: Щорічні оновлення специфікації, активна підтримка в браузерах, поява нових фреймворків

3. Роль CSS і JavaScript у сучасному вебі

Мова	**Призначення**
HTML	Структура вмісту
CSS	Візуальне оформлення
JavaScript	Поведінка, інтерактивність, логіка

Ці три мови — фундамент вебу. Неможливо стати фронтенд-розробником, не знаючи їх.

4. Синтаксис і основи CSS

CSS (Cascading Style Sheets — каскадні таблиці стилів) працює за дуже простою логікою: є HTML-елементи, до яких ми застосовуємо правила оформлення.

4.1 Структура CSS-правила

```
селектор {
  властивість: значення;
}
```

Наприклад:

```
h1 {
  color: red;
  font-size: 32px;
}
```

Це означає: знайди всі заголовки <h1> і зроби їх червоними, зі шрифтом 32 пікселі.

4.2 Типи селекторів

CSS дозволяє вибирати HTML-елементи різними способами:

- За тегом: p, div, h1
- За класом: .menu, .container (починається з крапки)
- За id: #header, #main (починається з гратки)
- Укладені селектори: div p — всі p всередині div
- Комбінації: ul li.active, nav a:hover

4.3 Основні властивості CSS

CSS має сотні властивостей, але ось найважливіші для початку:

Колір і фон:

```
color: blue;            /* Колір тексту */

background-color: yellow; /* Колір фону */

background-image: url("img.jpg");
```

Шрифти:

```
font-family: Arial, sans-serif;

font-size: 18px;

font-weight: bold;

text-align: center;

line-height: 1.5;
```

Розміри та блоки:

```
width: 300px;

height: auto;

padding: 20px;

margin: 10px;

border: 1px solid black;
```

Позиціювання:

position: static | relative | absolute | fixed | sticky;

top: 10px; left: 20px;

z-index: 10;

Відображення:

display: block | inline | inline-block | none | flex | grid;

visibility: visible | hidden;

4.4 Каскадність і специфічність

CSS розшифровується як “каскадні таблиці стилів” — це означає, що стилі можуть накладатись один на одного, і браузер вирішує, які саме застосувати.

Правила:

1. Важливіші стилі перекривають менш важливі.
2. Стилі ближчі до елемента (inline, у тегу) мають вищий пріоритет.
3. !important — завжди найвищий пріоритет (але краще не зловживати).

4.5 Селектори псевдокласів і псевдоелементів

Псевдокласи (працюють при взаємодії або станах):

```
a:hover       /* коли курсор наведений */

input:focus   /* коли поле вводу активне */

li:first-child /* перший елемент списку */
```

Псевдоелементи (вставляють новий вміст):

```
p::first-line    /* перша стрічка абзацу */

h1::before {

  content: "★ ";

}
```

4.6 Медіа-запити та адаптивність

Медіа-запити дозволяють змінювати стилі залежно від ширини екрана:

```
@media screen and (max-width: 768px) {

  body {

    background-color: lightblue;

  }

}
```

Це ключ до адаптивного дизайну, коли сайт добре виглядає і на комп’ютері, і на телефоні.

4.7 Flexbox і Grid — сучасна верстка

Flexbox — дозволяє розташовувати елементи в ряд чи стовпчик:

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Grid — дозволяє створювати сітки:

```
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 10px;
}
```

4.8 Як підключити CSS до HTML

Варіант 1: Вбудовані стилі

```
<style>
  p { color: green; }
</style>
```

Варіант 2: Зовнішній файл

```
<link rel="stylesheet" href="styles.css">
```

Варіант 3: Прямо в тегу (небажано)

```
<p style="color: red;">Привіт!</p>
```

5. Основи JavaScript: Синтаксис, логіка, робота з DOM

JavaScript — це мова програмування, яка виконується в браузері користувача і дозволяє створювати інтерактивні елементи на сторінці: спливаючі вікна, анімації, динамічні форми, оновлення даних без перезавантаження сторінки.

5.1 Підключення JavaScript до HTML

Варіант 1: У тілі HTML

```
<script>
  alert("Привіт, JavaScript!");
</script>
```

Варіант 2: Зовнішній файл

```
<script src="script.js"></script>
```

Рекомендовано використовувати зовнішні файли для кращої структури.

JavaScript — мова з динамічною типізацією

У мовах із динамічною типізацією не потрібно заздалегідь вказувати тип змінної. Тип визначається під час виконання коду, а не під час написання.

Приклад:

```
let value = 42;        // тип: number

value = "текст";       // тепер тип: string

value = true;          // тепер тип: boolean
```

JS дозволяє змінювати тип значення змінної на ходу. Це зручно, але може призводити до помилок, які складно виявити.

Плюси динамічної типізації:

- Можна швидко писати код
- Менше формальностей

Мінуси:

- Можна випадково передати “не той” тип
- Менше підказок від редактора
- Часті “дивні” помилки, як [] + {} = "[object Object]"

JavaScript — мова з прототипною системою

У JavaScript усе — об’єкти, і об’єкти можуть успадковувати властивості від інших об’єктів через прототипи.

Це не класична об’єктно-орієнтована модель (як у Java чи C#), де є класи й наслідування. У JavaScript — об’єкти створюються на основі інших об’єктів.

Приклад:

```
const person = {
  greet() {
    console.log("Привіт!");
  }
};


const student = Object.create(person);
student.study = function () {
  console.log("Я вчуся");
};


student.greet();  // Привіт!
student.study();  // Я вчуся
```

Об’єкт student не має свого методу greet, але він “успадкував” його від person через прототип.

Пояснення простими словами:

Прототип — це як шухляда, в яку JS заглядає, якщо не знаходить потрібної властивості в об’єкті. Якщо немає в цьому об’єкті — дивиться в прототип. Якщо й там немає — дивиться далі вверх по ланцюжку.

Сучасна альтернатива — класи (але все одно прототипи)

```
class Animal {
  speak() {
    console.log("Звук");
  }
}

class Dog extends Animal {
  speak() {
    console.log("Гав");
  }
}

const dog = new Dog();
dog.speak(); // Гав
```

Це виглядає як клас, але всередині все одно працює через прототипи.

Висновок:

- JavaScript динамічно типізований: змінні можуть міняти тип під час виконання
- JavaScript використовує прототипне наслідування, а не класичне
- Прототипи дозволяють організовувати спільну логіку між об’єктами без класів

5.2 Змінні

У JavaScript є три способи створити змінну:

```
var name = "Олег";    // старий спосіб (використовують рідко)
let age = 25;         // сучасний, змінне значення
const city = "Київ";  // константа, не можна змінити
```

5.3 Типи даних

Примітивні типи:

- String – рядки: "Привіт"
- Number – числа: 42, 3.14
- Boolean – логічні значення: true, false

- Undefined – значення не призначене
- Null – порожнє значення
- Symbol – унікальні ідентифікатори
- BigInt – великі числа

Складні типи:

- Object – об’єкти
- Array – масиви
- Function – функції

5.4 Оператори

- Арифметичні: +, -, *, /, %
- Присвоєння: =, +=, -=
- Порівняння: ==, ===, !=, <, >
- Логічні: &&, ||, !

5.5 Умовні конструкції

```
let age = 18;

if (age >= 18) {

  console.log("Повнолітній");
```

```
} else {
  console.log("Неповнолітній");
}
let color = "червоний";
switch (color) {
  case "зелений":
    console.log("Йди!");
    break;
  case "червоний":
    console.log("Стій!");
    break;
  default:
    console.log("Уважно!");
}
```

5.6 Цикли

```
for (let i = 0; i < 5; i++) {
  console.log("Крок " + i);
}
```

```
let i = 0;
while (i < 3) {
  console.log(i);
  i++;
}
```

5.7 Функції

Звичайна функція:

```
function sayHi(name) {
  console.log("Привіт, " + name + "!");
}
sayHi("Анна");
```

Стрілочна функція:

```
const add = (a, b) => a + b;
console.log(add(2, 3)); // 5
```

5.8 Масиви

```
let fruits = ["яблуко", "груша", "банан"];
console.log(fruits[0]);        // яблуко
```

```
console.log(fruits.length);    // 3

fruits.push("апельсин");       // додає в кінець

fruits.pop();                  // видаляє останній
```

5.9 Об’єкти

```
let user = {

  name: "Олег",

  age: 30,

  isAdmin: true

};

console.log(user.name);     // Олег

console.log(user["age"]);   // 30
```

5.10 Взаємодія з користувачем

```
alert("Це повідомлення!");

let name = prompt("Як тебе звати?");

console.log("Привіт, " + name);

let result = confirm("Тобі 18 років?");
```

5.11 Робота з DOM (Document Object Model)

Отримання елементів:

```
const title = document.querySelector("h1");

const button = document.getElementById("btn");
```

Зміна тексту і стилів:

```
title.textContent = "Новий заголовок";

title.style.color = "blue";
```

Реакція на події:

```
button.addEventListener("click", () => {

  alert("Кнопка натиснута!");

});
```

5.12 Таймери

```
setTimeout(() => {

  console.log("Пройшло 2 секунди");

}, 2000);

setInterval(() => {

  console.log("Кожні 3 секунди");

}, 3000);
```

5.13 Асинхронність

```
fetch("https://api.example.com/data")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Помилка", error));
```

Або:

```
async function getData() {
  const res = await fetch("https://api.example.com/data");
  const data = await res.json();
  console.log(data);
}
getData();
```

Що таке замикання (closure) в JavaScript

Замикання — це механізм у JavaScript, який дозволяє функції зберігати доступ до змінних з області, де вона була створена, навіть після того, як ця область виконання завершилася.

Інакше кажучи, функція “пам’ятає” те, що було навколо неї під час її створення.

Приклад простий:

```
function outer() {
  let name = "Олена";

  function inner() {
    console.log("Привіт, " + name);
  }

  return inner;
}

const greeting = outer(); // тут виконується outer, але inner не викликається
greeting(); // Привіт, Олена
```

У цьому прикладі:

- Функція outer створює змінну name і повертає функцію inner.
- Коли ми викликаємо outer(), вона повертає inner.
- Навіть після завершення outer(), функція greeting все ще має доступ до name — ось це і є замикання.

Навіщо це потрібно?

1. Інкапсуляція — ми можемо приховати змінні всередині функцій.

2. Фабрики функцій — створення функцій з “вшитими” параметрами.

3. Зберігання стану — наприклад, лічильник:

```
function createCounter() {
  let count = 0;


  return function () {
    count++;
    return count;
  };
}


const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Простими словами:

Замикання — це як чарівна коробка: ти кладеш у неї змінні, а потім інша функція може в будь-який момент відкрити цю коробку і дістати дані, навіть якщо зовнішня функція вже давно завершила роботу.

1. Приведення типів у порівнянні

```
console.log([] == []);          // false

console.log([] == ![]);         // true

console.log(null == undefined);  // true

console.log(null === undefined); // false

console.log('' == 0);           // true
```

Чому?

- [] == [] — різні об’єкти.
- [] == ![] → [] == false → '' == false → 0 == false → 0 == 0 → true
- == робить автоматичне приведення типів, а === — ні.

2. Додавання масивів і об’єктів

```
console.log([] + []);          // ""

console.log([] + {});          // "[object Object]"

console.log({} + []);          // 0  (в деяких випадках)
```

Чому?

- [] + [] → порожні рядки → ""
- [] + {} → "" + "[object Object]" → "[object Object]"
- {} + [] — залежить від того, як парсер трактує {}: як блок або об’єкт.

---

3. typeof null

console.log(typeof null); // "object"

Чому?

Це баг, який тягнеться ще з 1990-х. Виправити його неможливо без порушення зворотної сумісності.

---

4. Числа й рядки

console.log(1 + "2"); // "12"

console.log("2" + 1); // "21"

console.log("5" - 1); // 4

console.log("5" * "2"); // 10

Чому?

- Оператор + з рядком перетворює все на рядок.
- А -, *, / намагаються перетворити на числа.

---

5. Перевірка на NaN

```
console.log(NaN === NaN);       // false

console.log(isNaN("hello"));    // true

console.log(Number.isNaN("hello")); // false
```

Чому?

- NaN не дорівнює сам собі.
- isNaN перетворює аргумент у число, а тоді перевіряє.
- Number.isNaN перевіряє без перетворення.

---

6. Дивні булеві значення

```
console.log(Boolean([]));       // true

console.log(Boolean({}));       // true

console.log([] == false);       // true

console.log({} == false);       // false
```

Чому?

- Порожні об’єкти і масиви завжди truthy, але при порівнянні з == можуть вести себе непередбачувано.

7. Число з плаваючою точкою

console.log(0.1 + 0.2); // 0.30000000000000004

Чому?

Це не баг JavaScript, а особливість IEEE 754 — формату зберігання чисел у пам’яті.

8. Автоматичне створення глобальних змінних

```
function test() {
  a = 5;
}
test();
console.log(a); // 5
```

Чому?

Змінна a оголошена без let/var/const, тому автоматично стає глобальною. Це велика небезпека — завжди оголошуй змінні правильно!

9. Сортування масивів за замовчуванням

console.log([10, 1, 3].sort()); // [1, 10, 3]

Чому?

sort() сортує як рядки. Потрібен кастомний компаратор:

```
[10, 1, 3].sort((a, b) => a - b); // [1, 3, 10]
```

10. typeof функції

```
function hello() {}

console.log(typeof hello); // "function"
```

Чому?

Функції — це об’єкти, але typeof для них повертає "function" — єдиний тип, де typeof повертає щось специфічне, щоб було зручно.

У наступній частині: розбираємо CSS-препроцесори — Sass, SCSS, LESS: що це, як працює, коли використовувати.

Чудово! Ось четверта частина лекції — CSS-препроцесори. Це дуже корисна тема, яка допомагає ефективніше писати стилі, особливо у великих проєктах.

6. CSS-препроцесори: Sass, SCSS, LESS

6.1 Що таке препроцесор?

Препроцесор — це спеціальна мова, яка розширює можливості CSS і компілюється у звичайний CSS, зрозумілий браузеру. Препроцесори

допомагають організовувати код, використовувати змінні, вкладеність, міксини та інші корисні речі, яких бракує в «чистому» CSS.

6.2 Навіщо потрібен препроцесор?

Ось типові проблеми у CSS:

- Багато повторень: ті самі кольори, розміри, відступи в різних місцях
- Немає вкладеності: важко читати код
- Не вистачає модульності: складно розбити на частини
- Немає логіки: неможливо робити обчислення, умовні стилі тощо

Препроцесори вирішують усе це.

6.3 Sass / SCSS

Sass — найпопулярніший CSS-препроцесор. Має два синтаксиси:

- Sass (індентаційний):

```
$main-color: #333


body
  color: $main-color
```

- SCSS (як звичайний CSS, але з розширеннями):

```
$main-color: #333;


body {
  color: $main-color;
}
```

SCSS — найпоширеніший варіант, бо він майже не відрізняється від CSS.

6.4 Можливості SCSS

Змінні:

```
$primary-color: #3498db;
$padding: 10px;


button {
  background-color: $primary-color;
  padding: $padding;
}
```

Вкладеність:

```
nav {
```

```
  ul {
    list-style: none;
    li {
      display: inline-block;
      a {
        text-decoration: none;
      }
    }
  }
}
```

Міксини:

```
@mixin rounded($radius) {
  border-radius: $radius;
}


.box {
  @include rounded(10px);
}
```

Умови та цикли:

```
@for $i from 1 through 3 {
  .col-#{$i} {
    width: 100px * $i;
  }
}
```

6.5 LESS

LESS — ще один популярний препроцесор, схожий на SCSS, але менш гнучкий у великих проєктах.

Синтаксис LESS:

```
@main-color: #333;


body {
  color: @main-color;
}
```

Функції, вкладеність, міксини — усе є.

У 2010-х LESS активно використовували разом із Bootstrap, але з часом SCSS став більш стандартним.

6.6 Stylus (додатково)

Stylus — ще один препроцесор з дуже вільним синтаксисом (можна писати навіть без дужок і крапок з комою), але він менш популярний у спільноті.

6.7 Як скомпілювати SCSS у CSS

Є кілька варіантів:

1. Онлайн-компілятори

- sassmeister.com
- codepen.io

2. Використання CLI (терміналу)

sass styles.scss styles.css

3. Використання збирачів (Webpack, Gulp тощо) — розберемо в наступних розділах.

6.8 Переваги препроцесорів

- Менше повторів, більше логіки
- Легше підтримувати великі проєкти
- Краще структурувати код
- Більше можливостей, ніж у звичайному CSS

6.9 Недоліки (умовні)

- Потрібна компіляція перед переглядом
- Трохи складніше для новачків
- Може “переховати” помилки в стилях

У наступній частині: переходимо до трансляторів JavaScript — Babel, TypeScript, що вони роблять і навіщо потрібні.

Це дуже важлива тема, бо сьогодні майже кожен серйозний проєкт використовує хоча б один транслятор.

7. Транслятори JavaScript: Babel, TypeScript, інші

7.1 Що таке транслятор (transpiler)?

Транслятор (transpiler) — це інструмент, який перетворює код з однієї версії або мови в іншу. У світі JavaScript це переважно:

- Перетворення сучасного JS (ES6+) у старішу версію, сумісну з усіма браузерами
- Додавання типів і перевірок
- Перетворення альтернативних мов (наприклад, TypeScript, CoffeeScript) у звичайний JavaScript

7.2 Навіщо це потрібно?

- Не всі браузери підтримують нові можливості JS (особливо Internet Explorer)
- У великих проєктах хочеться мати типи, модулі, кращу структуру
- Деякі синтаксичні конструкції зручніші в нових версіях мови

7.3 Babel — найпопулярніший транслятор

Що таке Babel?

Babel — це інструмент, який перетворює сучасний JavaScript (ES6, ES7 і далі) у сумісний зі старими браузерами ES5-код.

Як це виглядає:

Було (ES6):

```
let name = "Іван";

const sayHi = () => console.log(`Привіт, ${name}`);
```

Стало (ES5):

```
var name = "Іван";

var sayHi = function () {

  console.log("Привіт, " + name);

};
```

Babel підтримує:

- Класи
- Стрілочні функції
- Async/await
- Модулі (import/export)
- JSX (у React)
- Плагіни для експериментальних фіч

7.4 TypeScript — надмножина JavaScript

Що таке TypeScript?

TypeScript — це мова, яка розширює JavaScript системою статичної типізації. Вона створена Microsoft і компілюється у звичайний JS.

Синтаксис:

```
let age: number = 25;

let name: string = "Олена";


function greet(user: string): string {

  return "Привіт, " + user;

}
```

Переваги TypeScript:

- Перевірка типів ще до запуску
- Підказки в редакторі коду (автозаповнення, IntelliSense)
- Зручна робота з великими командами
- Чітка архітектура проєктів

Недоліки:

- Потрібно компілювати у JS
- Трохи більший поріг входу
- Не всі бібліотеки мають типи

7.5 Інші транслятори

CoffeeScript

- Альтернативний синтаксис JavaScript
- Дуже лаконічний, схожий на Python
- Зараз майже не використовується

```
square = (x) -> x * x
```

Elm, PureScript, ReasonML

- Мови, які компілюються у JavaScript
- Фокусуються на функціональному програмуванні
- Рідко використовуються в продакшн-проєктах

7.6 Як використовують Babel і TypeScript у реальних проєктах?

- У React-проєктах — Babel + Webpack
- У Angular — все написано на TypeScript
- У Vue — підтримується як Babel, так і TypeScript
- У Next.js / Nuxt — можна обрати, що використовувати

7.7 Інструменти для роботи з трансляторами

1. Командний рядок

```
npx babel script.js --out-file compiled.js

tsc script.ts
```

2. Онлайн-редактори

- babeljs.io/repl
- typescriptlang.org/play

3. Збирачі (Webpack, Vite, Parcel) — ми про них говоритимемо у наступному розділі.

7.8 Порівняння Babel і TypeScript

Властивість	Babel	TypeScript
Типи	Ні	Так
Компілює у JS	Так	Так
Підтримка JSX	Так	Так
Навчальна крива	Низька	Середня
Підходить для	React, старі браузери	Великі проєкти, Angular

У наступній частині — автоматизатори і збирачі: Gulp, Webpack, Vite, Parcel — що це, навіщо потрібні, як працюють.

Чудово! Переходимо до шостої частини лекції — Автоматизатори та збирачі у веб-розробці. Це одна з найпрактичніших тем, бо саме тут починається «магія» професійної розробки.

8. Автоматизатори і збирачі: Gulp, Webpack, Vite, Parcel

8.1 Що це таке і навіщо?

У сучасній веб-розробці важливо не тільки написати HTML, CSS, JS, а й:

- стиснути код (мінімізація),
- скомпілювати SCSS у CSS або TypeScript у JS,
- об’єднати кілька файлів в один (конкатенація),
- оптимізувати зображення,
- перезбирати проєкт після змін.

Автоматизатори та білдери (збирачі) виконують ці завдання автоматично. Це:

- економія часу,
- стабільність збірки,
- готовність до продакшну.

8.2 Gulp — автоматизатор задач

Що це таке?

Gulp — це інструмент автоматизації, написаний на JavaScript. Він читає «рецепти» (називаються таски) і виконує їх послідовно або паралельно.

Що можна зробити за допомогою Gulp:

- компілювати SCSS у CSS

- мініфікувати CSS та JS
- оновлювати браузер при зміні файлів
- працювати з зображеннями
- створювати sprite-и

Схема роботи:

1. Встановлюємо Gulp:

```
npm install --save-dev gulp
```

2. Створюємо gulpfile.js:

```
const { src, dest, watch, series } = require('gulp');

const sass = require('gulp-sass')(require('sass'));


function compileSass() {

  return src('src/scss/**/*.scss')

    .pipe(sass())

    .pipe(dest('dist/css'));

}


exports.default = series(compileSass);
```

3. Запускаємо:

gulp

8.3 Webpack — модульний збирач

Що це таке?

Webpack — це збирач модулів, який об’єднує все (JS, CSS, зображення, шрифти, SVG) у один або кілька бандлів для використання у браузері.

Ключові поняття:

- Entry — звідки почати
- Output — куди зберегти результат
- Loaders — як обробляти різні типи файлів
- Plugins — додаткові дії (очистка, HTML, мініфікація)

Приклад налаштування:

```
// webpack.config.js
module.exports = {
  entry: './src/index.js',
  output: {
    filename: 'bundle.js',
    path: __dirname + '/dist'
  },
  module: {
```

```
    rules: [
      { test: /\.scss$/, use: ['style-loader', 'css-loader', 'sass-loader'] }
    ]
  }
}
```

Особливості:

- Webpack — потужний, але потребує конфігурації
- Працює з React, Vue, TypeScript, SCSS

8.4 Vite — надшвидкий збирач нового покоління

Що це таке?

Vite — це новий інструмент збірки, який використовує ES-модулі та має миттєву швидкість запуску навіть на великих проєктах.

Переваги Vite:

- блискавична швидкість
- підтримка TypeScript, SCSS, JSX з коробки
- гаряче оновлення (HMR)
- проста конфігурація

Приклад:

```
npm create vite@latest my-project
```

```
cd my-project

npm install

npm run dev
```

Підходить для:

- React, Vue, Svelte
- TypeScript
- сучасної frontend-розробки

8.5 Parcel — білдер без конфігурації

Що це таке?

Parcel — це білдер, який автоматично визначає типи файлів і що з ними робити. Його головна фішка — мінімум налаштувань.

Приклад:

```
npm install -g parcel

parcel index.html
```

Особливості:

- автоматичне оброблення JS, CSS, зображень
- інтегрований Babel і PostCSS
- швидкий старт

Інструмент	Тип	Гнучкість	Швидкість	Простота
Gulp	Автоматизатор	Висока	Середня	Середня
Webpack	Збирач модулів	Висока	Середня	Складна
Vite	Збирач	Середня	Дуже висока	Висока
Parcel	Збирач	Автоматичне	Висока	Дуже проста

8.6 Порівняння інструментів

8.7 Що обрати початківцю?

- Gulp — якщо хочеш зрозуміти, як все працює «вручну».
- Vite — якщо хочеш швидко стартувати сучасний проєкт (рекомендується).
- Parcel — якщо потрібна максимально проста альтернатива без конфігурації.
- Webpack — коли потрібно більше контролю (наприклад, для великих React-проєктів).

У наступній частині: підсумуємо все, що вивчили, і розглянемо стратегію вивчення фронтенду для новачків: які інструменти вчити спочатку, як будувати свою програму навчання, де практикуватися.

Чудово! Завершуємо нашу лекцію — ось сьома (остання) частина. У ній ми підсумуємо матеріал, дамо план навчання для початківця і поради, як рухатися далі.

9. Підсумок: CSS, JavaScript, інструменти розробника

Ми з тобою пройшли довгий шлях — від історії CSS і JavaScript до складних речей на кшталт трансляторів і збирачів. Зараз підведемо підсумки, щоб усе систематизувати.

9.1 Що ми вже знаємо

Тема	Що ти вивчив(ла)
CSS	Селектори, властивості, Flexbox, Grid
JavaScript	Змінні, типи, функції, DOM, події

Препроцесори	Sass, SCSS, LESS, вкладеність, міксини
Транслятори	Babel, TypeScript
Збирачі	Gulp, Webpack, Vite, Parcel

9.2 Як вивчати фронтенд у 2025

Крок 1: Основи HTML + CSS + JS

- верстай прості сайти (лендінги)
- використовуй Flexbox, Grid
- роби стилізацію за макетами (Figma)

Крок 2: Вивчи Git + GitHub

- створи акаунт
- вчи команди: git init, git add, git commit, git push

Крок 3: Освой препроцесори (SCSS)

- використовуй змінні, міксини, вкладеність
- підключай компілятор (наприклад, через Live Sass Compiler у VS Code)

Крок 4: Працюй з JavaScript

- роби невеликі проєкти: слайдер, todo-list, калькулятор
- обробляй події, міняй DOM

Крок 5: Навчися працювати з API

- використовуй fetch, async/await
- підключай публічні API (наприклад, погода, новини)

Крок 6: Збирай проекти

- спробуй Gulp або Vite
- структуруй папки: src, dist, assets, scss, js

Крок 7: Почни вивчати фреймворк (React або Vue)

- React краще для UI-логіки, Vue — для верстки
- використовуй Create React App або Vite

9.3 Де тренуватись і практикуватися

- codepen.io — швидке прототипування
- jsfiddle.net — онлайн-пісочниця
- freecodecamp.org — завдання і сертифікати
- frontendmentor.io — макети реальних сайтів
- github.com — твоя портфолійна платформа

9.4 Де брати проекти для практики

1. Лендінг сайт для вигаданого продукту
2. Сайт-візитка (для себе, друга, вчителя)
3. Інтерактивний список справ (todo)
4. Галерея зображень з фільтрацією
5. Калькулятор або конвертер валют

9.5 Як виглядає стек сучасного фронтенд-розробника

HTML → CSS (SCSS) → JavaScript → Git → Gulp/Vite → API → React → Node.js (базово)

10. Заключне слово

Навчання веб-розробці — це марафон, а не спринт. Не обов'язково знати все одразу. Головне — системно практикуватися і постійно застосовувати знання на реальних проектах. Помилки — це нормально. З кожним рядком коду ти стаєш сильнішим(ою).

> Не зупиняйся, навіть коли складно. Сайти, які змінюють світ, створюються тими, хто не здався.